SYSTEM ARCHITECTURE DOCUMENTATION

MCLUB CRM Framework

Park Zeman Chase Family Office

Complete technical reference for the MCLUB CRM system, covering the three-layer architecture (Presentation, Business Logic, Data Access), four-role access control model, ten database models, eight API route groups, and core business flows including order settlement, event management, and membership progression.

1. System Overview

MCLUB CRM (Customer Relationship Management System) is a comprehensive business management platform designed for Park Zeman Chase Limited, a Hong Kong-registered family office. The system serves as the digital backbone for managing client relationships, product offerings, order processing, commission distribution, and event coordination across four distinct user roles. Built with modern web technologies, the CRM provides role-based dashboards, real-time data analytics, and automated commission calculations to streamline the operations of a multi-stakeholder family office ecosystem.

The system architecture follows a three-tier model: a React-based frontend layer for interactive user interfaces, a Next.js API route layer for business logic and data processing, and a SQLite database layer managed through Prisma ORM for persistent data storage. This architecture ensures clear separation of concerns, rapid development cycles, and the ability to scale individual components as the business grows. The entire application runs as a single Next.js 16 deployment, leveraging server-side rendering for optimal performance and SEO capabilities while maintaining the flexibility of a client-side SPA for interactive dashboard features.

At its core, the MCLUB CRM implements a four-role access control model that mirrors the real-world organizational structure of the family office. MCLUB staff members have full administrative access to all data and operations, SME owners manage their product portfolios and track revenue, agents handle client relationships and monitor commissions, and end users interact with their purchase history and membership benefits. Each role sees a tailored dashboard with relevant metrics, navigation options, and action buttons, ensuring that users only access information pertinent to their responsibilities.

2. Technology Stack

The MCLUB CRM is built on a carefully selected technology stack that prioritizes developer productivity, runtime performance, and long-term maintainability. Each technology choice was made to address specific requirements of the family office domain, from handling multi-currency financial data to supporting Chinese-English bilingual interfaces. The following table provides a comprehensive overview of every technology component and its role within the system.

Layer	Technology	Version	Purpose
Framework	Next.js	16.1.1	Full-stack React framework with SSR/API routes
UI Library	React	19.0	Component-based UI with hooks and concurrent features

Layer	Technology	Version	Purpose
Styling	Tailwind CSS	4.x	Utility-first CSS with custom design tokens
Components	shadcn/ui + Radix UI	Latest	Accessible, composable UI primitives
Charts	Recharts	2.15	Responsive data visualization components
Animation	Framer Motion	12.x	Declarative animation and gesture handling
State	Zustand	5.x	Lightweight global state management
Forms	React Hook Form + Zod	7.6 / 4.0	Performant form validation with schema types
ORM	Prisma	6.11	Type-safe database access with migrations
Database	SQLite	3.x	Embedded relational database (dev.db)
Runtime	Bun	1.x	Fast JavaScript/TypeScript runtime
Language	TypeScript	5.x	Static type checking for reliability
Process	PM2	Latest	Production process manager with auto-restart
Proxy	Caddy	2.x	Reverse proxy with automatic HTTPS
VCS	GitHub	-	Source control and CI/CD pipeline

Table 1: Complete technology stack overview for the MCLUB CRM system.

3. Architecture Layers

The MCLUB CRM follows a layered architecture pattern where each layer has clearly defined responsibilities and communication boundaries. This separation ensures that changes to one layer do not cascade unpredictably through the system, enabling independent testing, optimization, and replacement of components. The three primary layers are the Presentation Layer (frontend), the Business Logic Layer (API routes), and the Data Access Layer (Prisma ORM + SQLite). Cross-cutting concerns such as authentication, authorization, and error handling are implemented as middleware and utility functions that span across layers.

3.1 Presentation Layer (Frontend)

The presentation layer is implemented as a single-page application (SPA) within Next.js, using a monolithic page.tsx component that dynamically renders different views based on the authenticated user role. The UI features a responsive sidebar navigation for desktop and a bottom navigation bar for mobile devices, ensuring seamless access across all screen sizes. The design system follows a dark theme with gold accent colors (#D4AF37), reflecting the premium positioning of the family

office brand. Key UI components include stat cards for KPI visualization, data tables with statusbadges, modal dialogs for data entry, and a timeline component for tracking client interactions overtime.

The frontend handles client-side routing through a custom state-based navigation system rather than Next.js file-based routing, which simplifies the role-based view switching mechanism. Each role sees a different set of navigation items: MCLUB_STAFF has access to six modules (Overview, Clients, Orders, Products, Commissions, Events), while END_USER sees a simplified four-module interface (Overview, My Products, Events, Membership Level). The presentation layer communicates with the API layer through a centralized apiFetch utility function that appends authentication parameters and handles timeout management with a 15-second abort controller.

3.2 Business Logic Layer (API Routes)

The business logic layer is implemented through Next.js API route handlers, organized under the /api directory with RESTful conventions. Each route handler is responsible for input validation, authorization checks, data transformation, and orchestration of database operations. A critical design decision was the implementation of sequential query execution in the dashboard API to prevent SQLite concurrent access crashes, as SQLite supports only one writer at a time. The API layer also handles complex business rules such as automatic commission calculation during order settlement, RSVP management for events, and timeline event generation for client activity tracking.

Authentication is handled through a simple email/password mechanism with bcrypt password hashing. The current implementation uses query parameter-based user identification (userId and userRole appended to API requests) rather than session-based or JWT-based authentication, which is appropriate for the demo phase but would need to be upgraded to a proper session management system for production deployment. The API follows a consistent response format with error objects containing descriptive messages and data objects containing the requested resources, making it straightforward for the frontend to handle both success and failure cases uniformly.

3.3 Data Access Layer (Prisma ORM)

The data access layer uses Prisma ORM to provide type-safe database interactions with automatic TypeScript type generation from the schema definition. The Prisma schema defines ten models with comprehensive relationship mappings, including self-referential User relations for the referral system and cascading deletes for event-related entities. The database connection uses SQLite with a connection limit of one (connection_limit=1) to prevent concurrent write errors inherent to SQLite architecture. The Prisma client is instantiated as a singleton through the db.ts utility module, preventing connection pool exhaustion in development mode where hot-reloading can create multiple client instances.

4. User Roles and Permissions

The MCLUB CRM implements a role-based access control (RBAC) system with four distinct user roles, each corresponding to a specific stakeholder in the family office ecosystem. The role hierarchy flows from MCLUB_STAFF (full access) down to END_USER (self-service access), with each role having precisely scoped permissions that limit data visibility and action capabilities to only what is necessary for that stakeholder type. This principle of least privilege ensures data security while maintaining operational efficiency.

Role	Description	Dashboard Modules	Key Permissions
MCLUB_STAFF	Internal staff with full admin access	Overview, Clients, Orders, Products, Commissions, Events	View all data, manage all records, settle ord
SME_OWNER	Product provider / business owner	Overview, My Products, Orders, Income, Events	View own products, track related orders, vie
AGENT	Business development agent / broker	Overview, My Clients, Commissions, Products, Events	Manage referred clients, track own commiss
END_USER	Final customer / member	Overview, My Products, Events, Membership	View owned products, RSVP to events, view mem

Table 2: User role definitions with dashboard modules and permission scope.

4.1 Demo Accounts

The system ships with four pre-configured demo accounts, one for each user role, allowing immediate exploration of all interface variations. These accounts are seeded automatically when a user first attempts to log in, triggered by a POST request to the /api/seed endpoint. All demo accounts share the password "demo123" for convenience during the demonstration phase. The following table lists each account with its associated role and primary use case for testing purposes.

Email	Role	Name	Purpose
kenneth@parkzeman.com	MCLUB_STAFF	Kenneth	Full admin testing and data management
calvin@mclub.com	SME_OWNER	Calvin	Product provider revenue tracking
agent@mclub.com	AGENT	Agent	Client management and commission tracking
user@mclub.com	END_USER	User	Customer purchase and membership experience

5. Database Schema

The MCLUB CRM database consists of ten Prisma models organized into two logical groups: core business models and event management models. The schema uses SQLite as the underlying

database engine with CUID-based primary keys for all entities. Relationships are enforced at thePrisma level through foreign key constraints, with cascade delete rules applied to event-relatedchild entities (tasks, budget items) to maintain referential integrity when events are removed. Thefollowing sections detail each model group with their fields, relationships, and business constraints.

5.1 Core Business Models

The core business models form the transactional backbone of the CRM system. The User model serves as the central entity, connecting to Clients through the agent relationship, to Products through the SME owner relationship, and to Commissions through the recipient relationship. The Order model acts as the nexus connecting Products, Clients, Users, and Commissions in a single transaction record. This interconnected design enables comprehensive revenue tracking from initial client referral through to final commission settlement, providing complete audit trails for all financial activities within the platform.

Model	Key Fields	Relationships	Business Purpose
User	email, name, role, password, phone, referralCode, isActive	Self-referral, Products, Agent Clients, Orders, Commissions, Events, RSVPs	Core user entity with role-based access and referral tracking
Client	name, phone, email, source, membershipTier, notes	Agent (User), Orders, TimelineEvents	Customer record with LTV tracking and member tier management
Product	name, category, description, keyBenefits, commissionRate, isActive	SME Owner (User), Orders	Investment product catalog with commission structure
Order	status, amount, currency, notes, productSnapshot, settledAt	Product, Client, Agent (User), Commissions, TimelineEvents	Transaction record with settlement tracking
Commission	role, amount, status	Order, Recipient (User)	Revenue share tracking per recipient and role

Table 3: Core business models with fields, relationships, and business purposes.

5.2 Event Management Models

The event management module was added as a significant feature extension to the core CRM, enabling MCLUB staff to organize networking events, seminars, dinners, workshops, and celebrations. The ClubEvent model serves as the parent entity with three child models (RSVP, EventTask, EventBudgetItem) connected through foreign keys with cascade delete rules. This design ensures that when an event is deleted, all associated RSVPs, tasks, and budget items are automatically cleaned up, preventing orphaned records. The RSVP model enforces a unique constraint on the (eventId, userId) pair, ensuring that each user can only RSVP once per event.

Model	Key Fields	Relationships	Business Purpose
ClubEvent	title, description, category, venue, eventDate, status, maxAttendees, budgetTotal	CreatedBy (User), RSVPs, Tasks, BudgetItems	Event entity with capacity and fee management
RSVP	status (PENDING/CONFIRMED/DECLINED/CHECKED_IN), guests, notes	ClubEvent, User	Attendance tracking with check-in workflow

Model	Key Fields	Relationships	Business Purpose
EventTask	title, status (TODO/IN_PROGRESS/DONE), priority, dueDate	Event, AssignedUser	Event preparation task delegation and tracking
EventBudgetItem	category, description, estimatedCost, actualCost	ClubEvent	Event budget planning with estimate vs actual tra
TimelineEvent	eventType, title, description	Client, Order (optional), CreatedByUser	Client interaction history for CRM timeline view

Table 4: Event management models with fields, relationships, and business purposes.

6. API Routes

The MCLUB CRM exposes a comprehensive RESTful API through Next.js route handlers, organized into eight functional groups. All endpoints accept and return JSON payloads, with authentication context provided through userId and userRole query parameters. The API follows REST conventions where GET requests retrieve resources, POST creates new resources, PATCH updates existing resources, and DELETE removes resources. The following table provides a complete reference of all available API endpoints, their HTTP methods, supported operations, and the roles that can access each endpoint.

Endpoint Group	Routes	Methods	Operations
Authentication	/api/auth/login, /api/auth/me	POST, GET	Login with email/password, get current user
Dashboard	/api/dashboard/[role]	GET	Role-specific KPI metrics and summary data
Clients	/api/clients, /api/clients/[id], /api/clients/[id]/timeline	GET, POST, PATCH	CRUD operations + timeline event management
Orders	/api/orders, /api/orders/[id], /api/orders/[id]/settle	GET, POST, PATCH	Order management + commission settlement
Products	/api/products	GET, POST	Product catalog with commission rules
Commissions	/api/commissions	GET	Commission records filtered by user role
Users	/api/users, /api/seed	GET, POST	User management + database seeding
Events (v1)	/api/events, /api/events/[id], /api/events/[id]/rsvp, .../checkin, .../tasks, .../budget	GET, POST, PATCH, DELETE	Full event CRUD, RSVP, check-in, tasks
Events (v2)	/api/e, /api/e/[id], /api/e/[id]/rsvp, .../checkin, .../tasks, .../budget	GET, POST, PATCH, DELETE	Mirror of v1 event API (shorter paths, same functi

Table 5: Complete API route reference with endpoint groups, methods, and operations.

7. Core Business Flows

7.1 Order and Commission Settlement Flow

The primary revenue flow in the MCLUB CRM follows a four-stage lifecycle: order creation, confirmation, completion, and settlement. When an agent refers a client to purchase a product, a new order is created with PENDING status. MCLUB staff then confirms the order, moving it to IN_PROGRESS. Upon service delivery, the order is marked COMPLETED. The final stage, settlement, triggers automatic commission calculation based on the product commission rules, distributing revenue shares to the agent, SME owner, and MCLUB according to the predefined percentage splits stored in the commissionRules JSON field of the Product model.

The commission calculation algorithm parses the JSON commission rules for the associated product, which specify three rates: agentRate (the percentage allocated to the referring agent), smeRate (the percentage allocated to the product provider/SME owner), and mclubRate (the percentage retained by MCLUB). These rates are applied to the order amount to generate individual Commission records for each recipient. Each commission record tracks its own status (PENDING or PAID), enabling the finance team to manage payout timing independently for each stakeholder. The order status is then advanced to SETTLED, and the commissionSettled flag is set to true, preventing duplicate settlement operations.

7.2 Event Management Flow

The event management module implements a complete event lifecycle from initial draft creation through to post-event completion. Events begin in DRAFT status, allowing MCLUB staff to configure all details including venue, capacity limits, registration fees, sponsorship information, and contact persons before publishing. Once published (PUBLISHED status), the event becomes visible to other user roles who can RSVP. The RSVP workflow supports four states: PENDING (initial signup), CONFIRMED (organizer approval), DECLINED (rejection), and CHECKED_IN (day-of-event attendance confirmation). The system enforces a maximum attendee constraint when specified, and tracks the number of guests each RSVP includes.

Parallel to the RSVP workflow, each event supports a task management system for event preparation. Tasks are assigned to specific users with priority levels (low, medium, high) and due dates, progressing through TODO, IN_PROGRESS, and DONE statuses. Additionally, a budget management module enables cost planning with estimated versus actual cost tracking across seven standard categories: venue, catering, decoration, AV, marketing, staff, and other. This comprehensive approach ensures that event organizers have full visibility into both the logistical preparations and financial commitments associated with each event, supporting the high-touch service expectations of the family office clientele.

7.3 Membership Level Progression

The CRM implements a four-tier membership system that segments clients by their engagement level and investment capacity. The entry-level Plan A provides basic access to the platform and product catalog. Plan B offers enhanced services with broader product access and priority event invitations. Plan C represents the premium tier with exclusive investment opportunities and personalized service. The highest tier, FULL membership, grants unrestricted access to all MCLUB services and events. Client progression through membership levels is tracked through the memberLevel enum field on the Client model, with the totalSpent field providing a quantitative basis for upgrade decisions. The timeline event system records membership changes, creating an audit trail of client lifecycle milestones.

8. Infrastructure and Deployment

8.1 Server Stack

The MCLUB CRM runs on a Linux-based server with the following infrastructure stack. The Next.js application runs in development mode on port 3000, managed by PM2 (Process Manager 2) which provides automatic restart on crashes, log management, and process monitoring. PM2 is configured to use the Bun runtime rather than Node.js, taking advantage of Bun significantly faster startup times and improved performance for TypeScript execution. The process is registered under the name "mclub-crm" in PM2, enabling straightforward management commands for status checks, restarts, and log viewing.

Caddy serves as the reverse proxy layer, forwarding requests from port 81 to the Next.js application on port 3000. Caddy was chosen for its automatic HTTPS certificate provisioning through Let Encrypt, minimal configuration requirements, and HTTP/2 support out of the box. The current configuration serves the application on the internal network, with the preview URL https://preview-chat-cfbf9474-2db8-4ba4-8247-31eed109e08e.space-z.ai/ providing external access through the Space-Z platform reverse proxy infrastructure. The source code is maintained in a GitHub repository at https://github.com/mypathtravel101-cyber/mclub, enabling version control, collaborative development, and automated deployment workflows.

8.2 CORS and Security Configuration

Cross-origin resource sharing (CORS) is configured through the Next.js next.config.ts headers function, which adds appropriate Access-Control-Allow-Origin headers for the preview domain. This configuration was necessary because the preview URL and the application server run on different origins, causing browsers to block API requests without proper CORS headers. The allowedDevOrigins configuration in next.config.ts also includes the preview domain to enable hot

module replacement during development. It is important to note that the Next.js middleware.ts filewas deliberately removed from the project, as it caused server crashes with Next.js 16 due toincompatibilities with the new App Router architecture. All cross-origin and authentication concernsthat would typically be handled in middleware are instead managed through the next.config.tsheaders configuration and the API route handler authentication logic.

8.3 Data Persistence

The application uses SQLite as its database engine, with the database file stored at prisma/dev.db. The DATABASE_URL environment variable is configured with a connection limit of one (file:./dev.db?connection_limit=1) to prevent concurrent write errors that are inherent to SQLite single-writer architecture. This limitation was discovered during development when the dashboard API attempted to execute multiple Prisma queries concurrently, resulting in SQLITE_BUSY errors. The fix involved reordering all dashboard queries to execute sequentially rather than in parallel, trading some performance for data integrity. For production deployment, migrating to PostgreSQL or MySQL would eliminate this concurrency limitation while preserving the Prisma ORM abstraction layer, requiring only a connection string change and database migration.

9. Known Limitations and Future Enhancements

The current implementation of the MCLUB CRM has several known limitations that should be addressed before production deployment. First, the authentication system uses query parameter-based user identification rather than secure session tokens or JWT, which exposes user IDs in server logs and browser history. Second, the SQLite database imposes a single-writer concurrency constraint that limits the system ability to handle simultaneous write operations from multiple users. Third, the frontend is implemented as a single large page.tsx component rather than a modular component architecture, making it difficult to maintain and test individual UI modules independently.

Future enhancements planned for the system include: implementing proper JWT-based authentication with HTTP-only cookies; migrating to PostgreSQL for production-grade concurrent access support; refactoring the frontend into separate page components per role using Next.js App Router file-based routing; adding real-time notifications via WebSocket for order status changes and event RSVP updates; implementing a full audit log system for regulatory compliance; adding multi-language support with next-intl for English and Chinese interfaces; and integrating with external payment gateways for automated order processing. The custom domain deployment to mclub.space-z.ai remains a platform-side configuration task that requires Space-Z infrastructure

team assistance to complete.